

Grounded Transport: A Proof-Relevant Rewriting Calculus for Lineage-Sensitive Systems

Author names withheld for review

First draft, September 21, 2026

Abstract

Ordinary rewriting records that one expression may replace another under an extensional relation. In lineage-sensitive systems this is not always enough: whether a substitution is warranted may also depend on the process that connects the terms, whether that process was independently grounded, whether its evidence has been maintained through time, and which obligations remain unresolved. We introduce a directed, proof-relevant transport calculus in which this warrant is carried as extractable data. Erasure, in two stages, recovers ordinary substitutability; we prove that erasure is sound, that it does not reflect grounding, and that distinct warrants can erase to the same judgement. Grounded-proper contexts transport witnesses compositionally. Instantiated on grounded seams, the calculus recovers an earlier seam-transport theorem, separates structural naturality from preservation of grounding, and yields located obstructions to substitution. Assessment is problem-indexed and three-valued (certified, refuted, open), retains every located failure, and does not treat missing evidence as refutation. A finite lineage audit is proved to reflect its specification. The development is mechanised in Coq without axioms in its kernel, and extraction to OCaml shows that certificates, lineage graphs and obstructions survive proof erasure. The calculus does not establish that declared records are complete or accurate; it makes precisely explicit what is being declared.

1 Introduction

1.1 The limitation of extensional rewriting

Ordinary rewriting licenses substitution under a relation. If $x R y$ and C is a context that respects the relevant relations, then $C[x] R' C[y]$. This is deliberately extensional: once $x R y$ has been established, the rewriting step normally does not retain how x and y were connected, which process produced the claimed agreement, whether one side was copied from the other, whether the evidence was independently grounded, whether the transformation has been maintained through time, or whether any obligation remains open.

For mathematical equality this forgetting is harmless, and it is the source of rewriting's power. For audited migrations, provenance-sensitive predicates, certified transformations and socio-technical assurance it is not always harmless. There, whether a substitution is *warranted* can depend on exactly the information that a relational proof discards.

1.2 The motivating phenomenon

Consider a dual-write migration in which two services record a status for each request, and a third source, an execution trace, records what actually happened. On an ordinary request all three agree. On one exceptional request both services report success—because one copied the other—while the trace records failure. The two service records agree with each other,

so ordinary relational rewriting between them succeeds; they disagree with the independently exhibited process. If we repair the disagreement by defining the trace valuation to be the first service’s value, all equations hold, but the “ground” now descends from the coordinate it was supposed to ground. The endpoint equation is true; its evidential warrant has failed. Section 2 works this example out completely; it is the separating model for most of the paper.

1.3 The proposed solution

We introduce *grounded transport*, written schematically $\text{GTrans}_c(x, y) : \text{Type}$ for a declared transport problem c . An inhabitant is not merely a proof that x and y are related: it carries the evidence by which transport is licensed. Erasure recovers ordinary substitutability in two stages,

$$\text{GTrans}_c(x, y) \xrightarrow{\text{erase}_1} \text{Cross}_c(x, y) \xrightarrow{\text{erase}_2} R(x, y),$$

first forgetting certificates while keeping the directed crossing, then forgetting the crossing while keeping the relation. The central asymmetry is

$$\text{GTrans}_c(x, y) \implies R(x, y), \quad \text{but in general} \quad R(x, y) \not\Rightarrow \text{GTrans}_c(x, y).$$

Ordinary rewriting is therefore the extensional image of the calculus, not something the calculus abandons.

1.4 Contributions

1. **A grounded transport calculus** (Section 4). A directed, proof-relevant transport judgement carrying extractable seam, exhibition, lineage, coverage and record evidence, with identity and composition, and with the algebraic laws stated modulo an explicit witness equivalence.
2. **Erasure theory** (Section 5). A two-stage erasure, its soundness, the induced preorder, a precise distinction between sound refinement and exact recovery, a proof that erasure does not reflect grounding, and a proof that distinct warrants can erase to the same judgement.
3. **Contextual preservation** (Section 6). A `GroundedProper` interface for contexts that transport witnesses compositionally, inducing ordinary `Proper` instances after erasure.
4. **Seam and temporal instances** (Sections 7 and 8). A compatibility result recovering an earlier grounded-seam theorem, and a separation of structural naturality, valuation naturality, preservation of grounding, and coverage.
5. **Obstruction theory** (Section 9). Located evidence showing why extensional or observational agreement can fail to warrant lineage-sensitive substitution.
6. **Evidence-bearing assessment** (Sections 10 and 11). A problem-indexed, three-valued semantics (certified, refuted, open) that retains every located failure, distinguishes missing evidence from counterevidence, and includes a lineage audit proved to reflect its specification.
7. **Mechanised validation** (Section 12). A Coq development, axiom-free in its kernel and checked with `coqchk`, with extracted OCaml programs showing that evidential data survive proof erasure.

We do not claim that equality, setoid rewriting, factorisation, or proof-relevant relations are new. The contribution is their integration into one calculus with the properties above.

1.5 Scope and non-claims

The calculus does *not* prove that a declared lineage graph is complete; that raw evidence is accurate; that a state model adequately represents a concrete process; that an institutional authority is legitimate or competent; that every transport obligation is decidable; or that ordinary rewriting is unsound. The claim is narrower: ordinary rewriting is *evidentially incomplete* for some applications, and this incompleteness can be repaired without giving up its extensional image. The calculus makes explicit what is declared; it does not make declarations true.

2 A minimal failure of extensional warrant

This section presents the motivating counterexample in full, before any general definitions. It is the model reused in Sections 5, 7 and 13.

2.1 The dual-write span

Let $\Sigma = \{r_{\text{ok}}, r_*\}$ be a set of requests and take $A = B = \Sigma$, with projections

$$A \xleftarrow{\text{back}} \Sigma \xrightarrow{\text{fwd}} B, \quad \text{back} = \text{fwd} = \text{id}.$$

Let both coordinate predicates be constantly true:

$$\Phi_A(\text{back}(r)) = 1, \quad \Phi_B(\text{fwd}(r)) = 1 \quad (r \in \Sigma).$$

Then $\Phi_A \circ \text{back} = \Phi_B \circ \text{fwd}$: pairwise, extensional seam coherence holds.

2.2 The independently grounded valuation

Let the seam carry its own valuation, exhibited from the execution trace:

$$\Phi_\Sigma(r_{\text{ok}}) = 1, \quad \Phi_\Sigma(r_*) = 0.$$

At r_* we have $\Phi_A(\text{back}(r_*)) = \Phi_B(\text{fwd}(r_*)) = 1$ but $\Phi_\Sigma(r_*) = 0$: the coordinates agree with each other while disagreeing with the exhibited process.

2.3 The copied alternative

Defining instead $\tilde{\Phi}_\Sigma = \Phi_A \circ \text{back}$ satisfies all three equations. But its lineage descends from the very coordinate it is supposed to ground, so it fails the lineage requirement (Section 11). Three things are thereby separated: equational agreement, independent grounding, and lineage warrant.

2.4 Requirements extracted from the example

1. Transport must retain an *exhibited crossing* (a seam element).
2. Grounding must refer to an *independent seam valuation*, not merely to the agreement of the two coordinates.
3. Lineage must be *data*, not an erased proposition.
4. Ordinary agreement must be *obtainable by erasure*.
5. Failure must report *why* grounding was unavailable.

The question the calculus answers is therefore:

What structure must be retained so that the copied and independently grounded transports do not collapse into the same formal object?

3 Formal setting and design choices

We fix only the notation needed later; the development is in Coq [11], and we write Coq identifiers in typewriter font.

3.1 Relations and ordinary rewriting

A *directed relation* on a type X is $R : X \rightarrow X \rightarrow \text{Prop}$. It is *reflexive* and *transitive* in the usual sense; we do not assume symmetry. A function $F : X \rightarrow Y$ is *proper* for relations R on X and S on Y when $R(x, y)$ implies $S(Fx, Fy)$; this is the reading of Coq’s **Proper** (respectful morphisms) in generalised rewriting [10]. We use only this much of setoid rewriting: what a context must respect.

3.2 Proof relevance

The distinction that matters is between

$$R(x, y) : \text{Prop} \quad \text{and} \quad \text{GTrans}(x, y) : \text{Type}.$$

A proposition can be inhabited only by indistinguishable proofs as far as extraction is concerned; a type-valued judgement can have several distinguishable inhabitants connecting the same endpoints. In Coq, proof fields are erased on extraction while data in **Type** or **Set** remain; we use this to place certificate *data* in **Type** and the proofs that the data satisfy their specifications beside them (Section 12).

3.3 Why transport is directed

A migration, derivation, update or maintenance step may be valid from x to y without licensing reversal. We therefore do not assume symmetry. The erased relation of Section 5 is a preorder, not necessarily an equivalence.

3.4 Why assessments are problem-indexed

Two things must be distinguished: the formal reflexivity of the transport relation, and the assessment of a proposed concrete transport problem. The former licenses formal self-substitution; it does not certify every concrete process whose endpoints happen to be equal. A state may have the same representation before and after a process while its provenance, calibration or authority has changed. Hence:

Relational identity is not an assurance certificate for an arbitrary identity-looking transition.

Assessments (Section 10) are consequently indexed by a declared transport *problem*, not merely by a pair of endpoints. This is not a technicality: a pair-indexed “refuted” would be false at reflexive pairs, where the identity witness needs no certificate.

3.5 The declared evidence boundary

Throughout, two questions are kept apart: whether declared records satisfy their stated conditions (which the kernel checks), and whether the declared records adequately represent the concrete world (which it cannot). The second is where construction and institutional obligations enter (Section 15).

4 The grounded transport calculus

4.1 The generic kernel

The kernel abstracts what is common to all instances: a proof-relevant, directed judgement with identity, composition, and a soundness map into an ordinary relation.

Definition 1 (Grounded transport structure). Let X be a type and R a relation on X . A *grounded transport structure* G for (X, R) consists of a type family $\text{GTrans}_G : X \rightarrow X \rightarrow \text{Type}$, an identity $\text{id}_x : \text{GTrans}_G(x, x)$, a composition $\text{GTrans}_G(x, y) \rightarrow \text{GTrans}_G(y, z) \rightarrow \text{GTrans}_G(x, z)$, and a soundness map $\text{GTrans}_G(x, y) \rightarrow R(x, y)$.

```
Record GTransport (X : Type) (R : X -> X -> Prop) : Type := {
  GT : X -> X -> Type;
  gt_refl : forall x, GT x x;
  gt_compose : forall x y z, GT x y -> GT y z -> GT x z;
  gt_sound : forall x y, GT x y -> R x y }.
```

Theorem 1 (Grounded reflexivity). $\text{id}_x : \text{GTrans}_G(x, x)$ for every x . (`grounded_transport_refl`)

Theorem 2 (Grounded composition). $\text{GTrans}_G(x, y) \rightarrow \text{GTrans}_G(y, z) \rightarrow \text{GTrans}_G(x, z)$. (`grounded_transport_compose`)

These are structure fields, so their proofs are immediate; their content is in the instances below. The word “identity” means the formal identity of the calculus, not a blanket certification of every temporally extended process with equal endpoints (Section 3).

4.2 Transport problems

A *transport problem* declares what is to be assessed. In the development a problem is a record `EvCandidate` with, in abridged form:

- a span $A \xleftarrow{\text{back}} \Sigma \xrightarrow{\text{fwd}} B$, coordinate valuations Φ_A, Φ_B , and an independent seam valuation Φ_Σ (all Boolean);
- observation maps M_A, M_B for the two coordinates;
- a declared finite lineage record (Section 11);
- a type of *maintenance evidence*, a type of located maintenance failures, and a function saying that a failure refutes any evidence;
- the same triple for *authority evidence*;
- summary interfaces for both (Section 12).

The endpoint relation is agreement of the coordinate valuations. The generic record imposes no institutional record format: maintenance and authority evidence are abstract types supplied by the application.

4.3 Certificates and grounded witnesses

The data-bearing components are records in `Type`.

Exhibition certificate. Follows the five conditions of an exhibited seam: a record identifier; stable, injective identifiers of seam elements; a declared process; the retained record together with an evaluator such that the seam valuation is a function of what the record retains (a factorisation obligation); a declared coverage interval; and a custodian.

Lineage certificate. Carries the finite lineage graph, a coverage record with its declared gaps, and the issued audit verdict as *data*, together with proofs that the graph is well-formed and satisfies conditions L1–L3 (Section 11).

Construction certificate. An inhabitant of the seam, an exhibition certificate, a lineage certificate *for the declared graph*, and the three-way grounding equations.

Certified transport. The observational (admissibility of both coordinates), construction, maintenance and authority evidence together.

```
Inductive RPath (c : EvCandidate) : rstate c -> rstate c -> Type :=
| rp_refl  : forall x, RPath c x x
| rp_step  : forall r (ct : CertifiedTransport c),
              RPath c (inl (back r)) (inr (fwd r))
| rp_trans : forall x y z, RPath c x y -> RPath c y z -> RPath c x z.
```

A *grounded transport witness* is an `RPath`: a path of crossings, each carrying a full certified transport. States are the disjoint sum $A + B$; a crossing at seam element r goes from `back(r)` to `fwd(r)`. This is the `GTrans` of a generic structure (`rich_structure`), whose soundness is extensional agreement of the coordinate valuations.

Composition is concatenation: each crossing keeps its own certificate, and there is no operation that merges or weakens evidence. Nothing is silently discharged, because witnesses exist only for discharged obligations; unresolved obligations live in assessments, not witnesses (Section 10).

4.4 Algebraic laws

`RPath` is a *free* structure, so identity and associativity do not hold by literal equality: `trans(refl, w)` is not syntactically w . We state them modulo the equivalence

$$w_1 \simeq w_2 :\iff \text{crossings}(w_1) = \text{crossings}(w_2),$$

where `crossings` flattens a witness to the list of its (seam element, certificate) pairs. This forgets bracketing and identity padding and *nothing else*: every seam element and every certificate is compared.

Theorem 3 (Witness algebra). $\simeq$ is an equivalence, composition respects it, and

$$\text{trans}(\text{refl}, w) \simeq w, \quad \text{trans}(w, \text{refl}) \simeq w, \quad \text{trans}(\text{trans}(w_1, w_2), w_3) \simeq \text{trans}(w_1, \text{trans}(w_2, w_3)).$$

Moreover $w_1 \simeq w_2$ implies equal extractable summaries. (`WEquiv_id_left`, `WEquiv_id_right`, `WEquiv_assoc`, `WEquiv_compose`, `WEquiv_summary`)

Consequently transport witnesses form a category *up to* $\simeq$ [7]; no quotient type is constructed, and we claim nothing stronger. Literal equality of composites is not claimed. This was checked before drafting: the laws hold only modulo $\simeq$, and the erased relation satisfies the ordinary preorder laws regardless (Section 5).

Remark 1. In the single-seam instance a non-identity witness is exactly one crossing (the path $A \rightarrow B$ has no continuation), so composition of non-trivial witnesses is degenerate there. Composition earns its keep through contexts and temporal evolution (Sections 6 and 8), not through long chains in one seam.

4.5 Packed endpoint-indexed warrants

Witnesses for different problems inhabit different indexed types. To compare warrants for the same endpoints we package the problem with the witness and an interpretation of its states

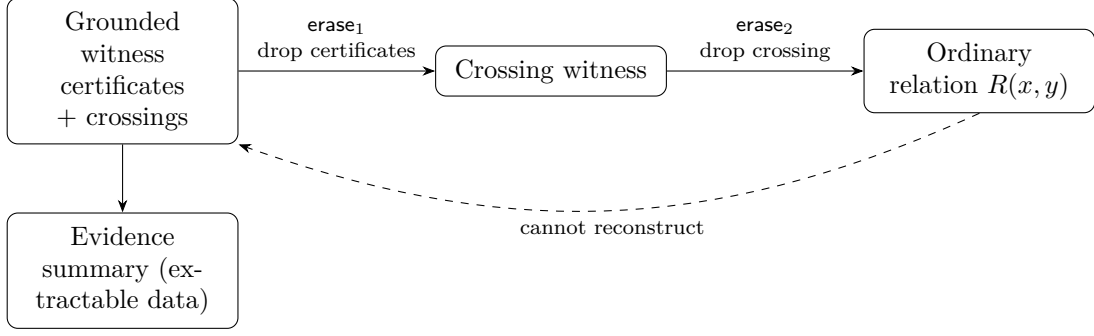


Figure 1: Two-stage erasure. The dashed arrow marks what erasure loses irreversibly (Theorems 6 and 7). The evidence summary is a separate, lossy projection that keeps extractable identifiers.

Table 1: Grounded versus ordinary rewriting.

	Ordinary rewriting	Grounded transport
Judgement	$R(x, y) : \text{Prop}$	$\text{GTrans}_c(x, y) : \text{Type}$
Inhabitants	Indistinguishable proofs	Distinguishable witnesses
Carries seam / lineage / coverage	No	Yes, as data
Contexts	Proper : preserves the relation	GroundedProper : maps witnesses
Failure	Absence of proof	Located obstruction, or open obligation
Relation to the other	—	Erases to it; not conversely

into a common endpoint type E :

$$\text{PackedWarrant}(x, y) = \sum_{c:\text{Problem}} \sum_{s,t} (\text{emb}_c(s) = x) \times (\text{emb}_c(t) = y) \times \text{GTrans}_c(s, t).$$

This places warrants from problems with *different* declared lineage into one collection without pretending their lineage records coincide. Erasure of a packed warrant lands in the single proposition $\text{ErasedAt}(x, y)$, which does not record which problem or witness was used (PackedWarrant , erase_packed).

5 Erasure

5.1 Two stages

Erasure forgets in two stages, and both are lossy.

First erasure: certificates to crossings. $\text{Cross}_c(x, y)$ is the type of *crossing witnesses*: paths of grounded crossings, each crossing carrying the two grounding equations $\Phi_A(\text{back}r) = \Phi_\Sigma(r)$ and $\Phi_B(\text{fwd}r) = \Phi_\Sigma(r)$ but no certificate. The map $\text{erase}_1 : \text{GTrans}_c(x, y) \rightarrow \text{Cross}_c(x, y)$ discards exhibition, lineage, coverage, maintenance and authority evidence while keeping the directed path (rpath_forget).

Second erasure: crossings to relation. $\text{erase}_2 : \text{Cross}_c(x, y) \rightarrow R(x, y)$ maps a crossing path to the extensional agreement of the coordinate valuations at its endpoints (opath_sound). The composite $\text{erase} = \text{erase}_2 \circ \text{erase}_1$ yields ordinary substitutability.

5.2 Soundness and the erased preorder

Theorem 4 (Erasure soundness). *If $w : \text{GTrans}_c(x, y)$ then $R(x, y)$, where R is agreement of the coordinate valuations. (erase_sound, rpath_sound; generically erasure_sound)*

For a generic structure G define the *erased relation* $\text{erased}_G(x, y) \equiv \text{inhabited}(\text{GTrans}_G(x, y))$.

Theorem 5 (Erased preorder). *erased_G is reflexive and transitive, and is contained in R . (erased_PreOrder, erasure_sound)*

Since composition holds up to $\simeq$ (Theorem 3) but the erased relation only records inhabitation, the erased relation satisfies the ordinary preorder laws with no qualification.

5.3 Sound refinement versus exact recovery

One must be careful with the word “recovery”. If the erased relation *is defined* as the image of grounded transport, then $R_G(x, y) \iff \exists w : \text{GTrans}(x, y)$ holds by definition. If instead one starts from an independently defined relation R , the general result is only the soundness map $\text{GTrans}(x, y) \rightarrow R(x, y)$. The converse,

$$R(x, y) \implies \|\text{GTrans}(x, y)\|,$$

requires a completeness (representability) hypothesis $\text{Complete}(G)$, under which R and erased_G coincide (erasure_exact_of_complete); it holds for the trivial structure whose witnesses are the proofs themselves (preorder_complete). It fails in precisely the cases of interest. We therefore call the generic result a *sound refinement*, not an equivalence with all ordinary rewriting.

5.4 Non-reflection

Theorem 6 (Erasure does not reflect grounding). *There exist a transport problem c and states x, y with $R(x, y)$ and $\neg \text{erased}(x, y)$. (erasure_not_reflecting, copied_ungrounded_agreement)*

Instance. Take the dual-write problem of Section 2, states $x = \text{inl}(r_*)$ and $y = \text{inr}(r_*)$. Both coordinate valuations are true, so $R(x, y)$. Every crossing requires a certified transport, and a certified transport would include the grounding equations at r_* , which fail; the declared lineage independently fails L1. So the problem admits no certified transport, and every witness is the identity path; since $x \neq y$, none connects them. $\square$

The same fact is a generic *ungrounded agreement*: extensional agreement without a transport witness, whose existence refutes Complete and hence exact recovery.

5.5 Same endpoints, different warrants

Theorem 7 (Warrant multiplicity). *There exist $w_1, w_2 : \text{PackedWarrant}(x, y)$ for the same endpoints such that both erase into $\text{ErasedAt}(x, y)$, while their extractable summaries differ, and hence $w_1 \neq w_2$. (packed_warrants_same_endpoints; within one problem same_endpoints_diff_warrants)*

The witnesses in our instance differ in seam element, exhibition record, custodian, coverage interval, authority, and—across problems—declared lineage dispositions. We do not assert equality of the two erased *proofs*: ErasedAt is a proposition, so equality of its proofs would require proof irrelevance and would be uninformative. The content is that erasure maps both witnesses into one proposition that records nothing about which was used, while the summary separates them. This is the most direct proof that ordinary rewriting forgets evidential structure.

6 Grounded-proper contexts

6.1 Definition

A context $F : X \rightarrow Y$ is *grounded-proper* between structures G_X, G_Y when it maps witnesses functorially:

$$\text{map}_F : \text{GTrans}_{G_X}(x, y) \rightarrow \text{GTrans}_{G_Y}(Fx, Fy),$$

with $\text{map}_F(\text{id}) = \text{id}$ and $\text{map}_F(v \circ w) = \text{map}_F(v) \circ \text{map}_F(w)$. This is stronger than `Proper`, which maps only proofs of a relation: `GroundedProper` preserves the *warrant* for the relation. (`GroundedProper`)

6.2 Identity and composition

Theorem 8 (Grounded-proper composition). *The identity context is grounded-proper, and if F and G are grounded-proper then so is $G \circ F$. (`gp_id`, `gp_comp`, `grounded_proper_compose`)*

6.3 Nested contextual preservation

Theorem 9 (Contextual preservation). *If $w : \text{GTrans}(x, y)$ and $C_1, \dots, C_n$ are grounded-proper, then*

$$\text{map}_{C_n} \cdots \text{map}_{C_1}(w) : \text{GTrans}(C_n[\cdots C_1[x]], C_n[\cdots C_1[y]]),$$

and the witness map of a composite context is the composite of witness maps. (`contextual_preservation`, `contextual_preservation_nested`)

6.4 Erased Proper

Every grounded-proper context induces an ordinary proper instance for the erased relation, `Proper(erasedGX  $\Rightarrow$  erasedGY)  $F$` (`grounded_proper_erased`, `grounded_proper_Proper`). The converse is *not* claimed: a context may preserve endpoint equivalence while discarding or fabricating lineage, so an ordinary `Proper` instance does not yield a witness map.

6.5 Relation to rewriting tactics

The development provides semantic combinators and instances, not an automated witness-carrying rewriting tactic. This separates the semantic kernel from later proof engineering: a tactic consuming the kernel should fail only when no suitable witness was supplied or constructed, never because the semantics needed weakening.

7 Grounded seams as a canonical instance

7.1 Spans, valuations and three-way grounding

A span $A \xleftarrow{\text{back}} \Sigma \xrightarrow{\text{fwd}} B$ carries Boolean valuations Φ_A, Φ_B and a seam valuation Φ_Σ . The *grounding equations* require both coordinate valuations to agree with the independently exhibited seam valuation:

$$\Phi_A(\text{back}(r)) = \Phi_\Sigma(r), \quad \Phi_B(\text{fwd}(r)) = \Phi_\Sigma(r) \quad (r \in \Sigma).$$

Contrast *extensional seam agreement*, which is only pairwise: $\Phi_A(\text{back}(r)) = \Phi_B(\text{fwd}(r))$.

The definitions here are those of an earlier, separately published supplement, which we preserve byte-for-byte and build as an independent library (Section 12). Our compatibility layer takes states to be $A + B$ and defines crossing witnesses as paths of crossings, where a crossing at r carries *both* grounding equations against Φ_Σ . An earlier reconstruction of this

instance used only the pairwise equation; it collapsed exactly the distinction this paper is about, and is retained only as a clearly labelled weaker layer.

7.2 Grounding erases to extensional coherence

Theorem 10 (Grounding erases). $\text{GroundingEquations} \implies \forall r, \Phi_A(\text{back}(r)) = \Phi_B(\text{fwd}(r))$.
(original_grounding_erases_to_extensional)

The proof goes through the generic calculus: the equations supply a crossing at every seam element, and erasure yields pairwise agreement.

7.3 Non-converse

Theorem 11 (Pairwise agreement does not imply grounding). *There exist a span and valuations with pairwise agreement at every seam element but no grounding.* (pairwise_agreement_not_grounding; from copied_extensionally_coherent and copied_not_grounded)

The witness is the dual-write example. Its generic reading is Theorem 6: the legacy counterexample embeds as an instance of ungrounded agreement (original_copied_counterexample_embeds), and, conversely, the legacy copied_not_grounded is recovered from the generic obstruction (copied_not_grounded_from_generic).

7.4 Extensional factor transport

We state the weak theorem first. Let $M_A : A \rightarrow O_A$ and suppose $\Phi_A = h \circ M_A$.

Theorem 12 (Extensional factor transport). *If extensional seam agreement holds, then $\Phi_B(\text{fwd}(r)) = h(M_A(\text{back}(r)))$ for every r .* (extensional_factor_transport)

7.5 Grounded factor transport

Corollary 13 (Grounded factor transport). *If the grounding equations hold and $\Phi_A = h \circ M_A$, then $\Phi_B(\text{fwd}(r)) = h(M_A(\text{back}(r)))$.* (grounded_factor_transport; in exactly the legacy statement, original_seam_transport_from_generic)

The route is $\text{GroundingEquations} \Rightarrow \text{ExtensionalAgreement} \Rightarrow \text{factor transport}$. Both theorems have the *same conclusion*; the grounded one has stronger premises. The interpretation is the point of this paper in miniature:

Grounding does not change the endpoint equation. It changes the warrant carried with the use of that equation.

This is not a weakness to be hidden. The endpoint equation can remain true after the warrant supporting it has failed (extensional_conclusion_survives_ungrounded); that is exactly what non-reflection expresses. A reviewer who observes that the earlier theorem uses stronger premises than its conclusion needs is correct, and the observation is part of the contribution.

7.6 Compatibility

The unchanged earlier supplement instantiates the generic calculus, and its transport theorem is recovered from it with the statement unchanged. This establishes continuity and makes clear which material is new: the generic calculus, erasure, contexts, obstruction, and assessment. The earlier supplement's files are checksum-verified in the build (Section 12).

8 Dynamic grounded transport

The synchronic grounding of Section 7 concerns one stage. This section asks what survives when the system evolves. The answer separates four notions that are easily conflated: structural naturality, valuation naturality, preservation of grounding, and coverage.

8.1 Structural seam evolution

A seam evolution from stage t to u consists of maps $U_{t,u}^\Sigma$, $U_{t,u}^A$, $U_{t,u}^B$ on the seam and the two coordinates such that the squares commute:

$$\text{back}_u \circ U_{t,u}^\Sigma = U_{t,u}^A \circ \text{back}_t, \quad \text{fwd}_u \circ U_{t,u}^\Sigma = U_{t,u}^B \circ \text{fwd}_t.$$

Evolutions have identity and composition. We recover both from the earlier supplement, and show that the state map of a composite evolution agrees pointwise with the composite of state maps (`evo_state_id`, `evo_state_compose`), so the composite evolution and `gp_comp` of the steps act identically.

8.2 Structural versus valutional naturality

The commuting squares say nothing about valuations. If the value spaces carry a transport $U_{t,u}^V$, one may additionally require all three valuations to be natural:

$$\Phi_{A,u} \circ U_{t,u}^A = U_{t,u}^V \circ \Phi_{A,t}, \quad \Phi_{B,u} \circ U_{t,u}^B = U_{t,u}^V \circ \Phi_{B,t}, \quad \Phi_{\Sigma,u} \circ U_{t,u}^\Sigma = U_{t,u}^V \circ \Phi_{\Sigma,t}.$$

We do not require the values to stay unchanged: a dynamic predicate may change while remaining coherently transported. Requiring $U^V = \text{id}$ would turn a dynamic predicate into a temporal invariant.

8.3 Preservation on the image

Define `PreservesGroundingOnImage(U)`: for every seam element r , grounding at t implies grounding at $U_{t,u}^\Sigma(r)$. It concerns only seam elements of the form $U_{t,u}^\Sigma(r)$, and says nothing about target elements outside the image.

Theorem 14 (Naturality preserves grounding on the image). *If the structural squares commute and all three valuations are natural with respect to some U^V , then `PreservesGroundingOnImage(U)`. In particular, if grounding holds at t it holds at every transported seam element at u . (`natural_valuations_preserve_grounding_on_image`, `grounding_at_transported_elements`)*

Naturality also composes, with U^V composing (`natural_id`, `natural_compose`). A `SeamEvolution` that preserves grounding on the image is a grounded-proper context (`original_grounding_preserving_evolution_is_grounded_proper`); the hypothesis is exposed in the name because the witness map must send a grounded crossing at r to a grounded crossing at its image.

8.4 Coverage

Theorem 15 (Global preservation under coverage). *If $U_{t,u}^\Sigma$ is surjective, grounding is preserved on its image, and grounding holds throughout the seam at t , then grounding holds throughout the seam at u . (`surjective_evolution_preserves_all_grounding`)*

Without surjectivity, new target seam elements require new grounding evidence. We exhibit a model in which preservation on the image holds and every stage- t element is grounded, yet a new stage- u element outside the image is not (`preservation_without_coverage`). This separates *continuation* from *coverage*, and fits the intended reading: a new process requires a new certificate.

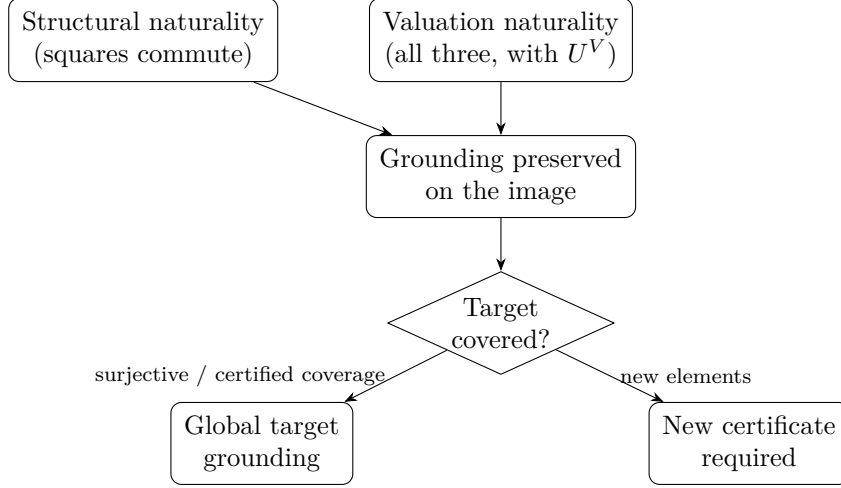


Figure 2: Structural and valutional preservation. Structure alone does not give preservation of grounding; preservation alone does not give coverage.

8.5 Dynamic failure

Theorem 16 (Extensional persistence without grounded preservation). *There is an evolution for which the structural squares commute; extensional agreement holds at both stages; grounding holds at t ; and grounding fails at the transported element at u . Ordinary factor transport remains available at u . (dynamic_failure_visible, ordinary_rewriting_still_succeeds, structural_naturality_not_preservation)*

The model has two stages and one seam element. The coordinates read true at both stages, so pairwise agreement persists. The seam valuation is true at the first stage and false at the second. No evolution preserves grounding, because the unique transported element loses it. This is the formal maintenance-debt example: warrant was valid at issuance, the system evolved, and the coordinates continue to agree while the independently evolved ground no longer does.

8.6 Interpretive consequence

Structural naturality preserves the relationship between carriers and projections. It does not, by itself, preserve the warrant attached to their valuations. A migration whose squares commute can be structurally faithful and evidentially stale.

9 Obstructions to substitution

9.1 Fibre obstruction

Let $M : X \rightarrow O$ be an observation map and $\Phi : X \rightarrow V$ a predicate. Φ factors through M iff it is constant on the fibres of M ; this is the established factorisation criterion, a set-theoretic form of the Doob–Dynkin lemma and of the exactness condition of abstract interpretation [5], and we use it as such. A pair x, y with $M(x) = M(y)$ and $\Phi(x) \neq \Phi(y)$ certifies that Φ cannot be determined from the observations alone (`fibre_obstruction`). The direction from factorisation to fibre-constancy and the relational converse are constructive (`factors_constant`, `constant_relational`); the finite converse needs a complete enumeration and decidable equality (`finite_constant_factors`); the unrestricted functional converse needs classical choice and is isolated (Section 12).

9.2 Lineage-sensitive obstruction

Theorem 17 (Observational agreement cannot license lineage-sensitive substitution). *If $M(x) = M(y)$ and a predicate L has $L(x) \neq L(y)$, then $\neg \text{Proper}(\sim_M, =)(L)$. Moreover, if grounded transport is sound for agreement of L , then no transport witness connects x and y . (`not_proper_of_distinguished`, `observational_insufficient`, `observational_not_grounded`)*

The instance is the dual-write example: the copied value is identical on both requests, yet the exhibited trace status separates them, so the trace status is not determined by the copied value (`trace_not_determined_by_copied_value`). The stronger, grounded reading is that observational equivalence $M(x) = M(y)$ does not imply $\text{GTrans}(x, y)$.

9.3 Located obstruction data

Failures are represented as data that names the defect, not as a Boolean. The construction obligation can fail by: an empty seam; a seam element on which a coordinate valuation departs from the seam valuation; a defeated exhibition; a lineage descent (an ancestor path from a coordinate to the ground); an undisclosed shared ancestor (naming the node); the absence of an independent claim-relevant source; or a malformed lineage record. The observational obligation fails by a *fibre witness* (a pair of states). The maintenance obligation, for temporal systems, fails by either the absence of any structural evolution or, for every evolution, a named seam element at which grounding is lost. Authority failure is a type supplied by the application.

Each located failure comes with a proof that it refutes the corresponding evidence, so a failure is not merely a label but a counter-argument (`located_refutes`).

9.4 Obstruction versus absence of evidence

A located obstruction warrants rejection. A missing record may instead leave the assessment open. A missing lineage record, a lineage record with declared coverage gaps, and an incomplete finite search are all *unresolved*, not refuted; only a proved defect refutes. Keeping these apart is what allows the assessment of Section 10 to avoid collapsing three different situations into one negative answer.

10 Evidence-bearing transport assessment

10.1 Component verdicts

For each obligation an assessment is one of three things: a positive certificate, located counterevidence, or an identification of what is missing.

$$\text{Assessment}(P) = \text{Discharged}(e) + \text{Failed}(d) + \text{Unresolved}(o),$$

where e is positive evidence, d is located counterevidence, and o names the missing evidence or decision. In the development, unresolved reasons include a not-yet-assessed obligation, a missing lineage record, a coverage gap (naming the gaps), an incomplete search, and an undeclared policy.

10.2 Four diagnostic loci

The obligations instantiate four loci: *observation* (the valuation factors through the observations), *construction* (an exhibited seam with independently grounded lineage), *maintenance* (warrant preserved through evolution), and *institution* (a declared rule or authority licenses the transformation). They are diagnostic loci, not a partition: failures can coexist.

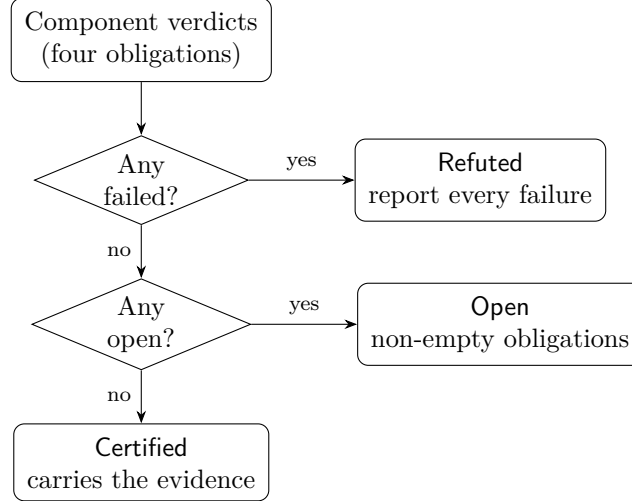


Figure 3: Verdict aggregation. Verdicts are singular; the *report* behind a verdict retains every failed and every open obligation (Theorem 18).

10.3 The assessment type

For a transport problem c ,

```

Inductive TransportAssessment (c : EvCandidate) : Type :=
| Certified : CertifiedTransport c -> TransportAssessment c
| Refuted   : Obstructions c       -> TransportAssessment c   (* head :: tail *)
| Open      : list OpenObligation -> TransportAssessment c.

```

$\text{Certified}(w)$ carries the evidence warranting transport; $\text{Refuted}(os)$ carries a *non-empty* collection of located obstructions; $\text{Open}(ds)$ names the unresolved obligations. The assembly function combines the four component verdicts.

10.4 Reports versus verdicts

The aggregate verdict and the complete report are different things and we keep them apart. The verdict follows the componentwise rule: no failure and no open obligation gives **Certified**; at least one failure gives **Refuted**; no failure but at least one open obligation gives **Open** (as `semble_status`). The *report* lists every obligation in exactly one of three lists—failed, open, discharged—whatever the verdict.

Theorem 18 (Complete reporting). *The verdict of the report is the verdict of the assembled assessment, and every obligation is accounted for exactly once: $|failed| + |open| + |discharged| = 4$. (`report_verdict_agrees`, `report_accounts_for_all`)*

The distinction matters. The **Refuted** constructor itself carries the located failures; if a failure co-occurs with an unresolved obligation, the verdict is **Refuted** but the open obligation must not vanish because another obligation already refutes transport. It does not: it remains in the report (`refuted_verdict_keeps_open_item`, where a copied ground fails construction while maintenance is unassessed). This was a gap in an earlier form of the development, in which the assembled **Refuted** value dropped co-occurring open items; it is repaired by separating the report from the verdict.

10.5 Soundness and characterisation

Theorem 19 (Certification soundness). *A certified assessment carries certified transport, and that erases to the legacy Transportable predicate. An assembled assessment is **Certified** exactly*

when every obligation is discharged. (`certified_gives_transportable`, `assemble_status`)

Theorem 20 (Refutation soundness). *If o is a collection of located obstructions for c , no certified transport for c exists, and this holds of every member of the collection.* (`obstructions_refute`, `each_obstruction_refutes`)

Theorem 21 (Open transport). *An assembled assessment is **Open** iff no obligation has located counterevidence and at least one obligation is unresolved; it neither asserts transportability nor its negation. Moreover an **Open** assessment always names at least one unresolved obligation.* (`assemble_status`, `underdetermined_iff`, `assemble_open_nonempty`)

Theorem 22 (Complete failure reporting). *A refuted assembly reports every failed obligation, in a fixed order (observational, construction, maintenance, institutional).* (`assemble_refuted_reports_all`)

Thus assessment is *cumulative* rather than precedence-based: in a model where a copied ground and a lost grounding coexist, both obstructions appear (`both_obstructions_reported`). Verdicts may be singular, but warrant failures are cumulative.

10.6 Decidable specialisation

When every obligation has a decision procedure the open case disappears: the status is then never **Open**, and **Certified** (resp. **Refuted**) coincides with transport (resp. its negation) for the conjunction of the obligations (`decidable_not_underdetermined`, `decided_certified_iff_transport`, `decided_refuted_iff_not_transport`). A Boolean classifier is thereby a *specialisation* of the evidence-sensitive assessment, not its foundation. General admissibility, institutional adequacy, and real-world lineage completeness are not assumed decidable. The manuscript-level status “warrant conflict” cannot arise here, since one assessment of one obligation is exactly one of the three cases.

11 Finite lineage certificates

11.1 Data model

A *lineage record* is finite data: the derived fields with their parents; the raw inputs; the claim-relevant raw inputs; three distinguished nodes d_A , d_B and g (the determinations $\Phi_A \circ \text{back}$, $\Phi_B \circ \text{fwd}$ and Φ_Σ); and a disposition (justified, or open defeater) for shared ancestors. It is represented as lists of numbers so that it survives extraction.

11.2 Lineage conditions

Write $\text{anc}(x)$ for the strict ancestors of x . The valuation Φ_Σ is *lineage-grounded* when:

L1, no descent: $d_A \notin \text{anc}(g)$ and $d_B \notin \text{anc}(g)$;

L2, disclosure: every shared non-raw ancestor of d_A and d_B carries a disposition;

L3, independent source: the ground-only ancestry contains a claim-relevant raw input.

Well-formedness is part of the requirement: unique names, declared parents, distinguished nodes derived, relevant inputs raw, and acyclicity.

11.3 Coverage

A passing verdict applies only to the declared graph *and* its declared coverage. A partial graph must not be reported as globally lineage-grounded; the record carries its declared gaps, and a passing graph with non-empty gaps is coverage-qualified, hence open, not accepted.

11.4 A checked decision procedure

We define the audit as a function `lineageCheck(g, c)` over a lineage record g and coverage record c , returning *accepted*, *rejected with a located defect*, or *open with the gaps*. Ancestors are computed by iterating a de-duplicating closure step as many times as there are nodes; the correctness argument is a pigeonhole one and needs only that every parent is a declared node.

Theorem 23 (Ancestor correctness). *ancestors is sound, and complete when parents are declared: $m \in \text{ancestors}(n) \iff m$ is a strict ancestor of n . (`ancestors_sound`, `ancestors_complete`, `ancestors_iff`)*

Theorem 24 (Lineage-check reflection). *Unconditionally,*

$$\text{lineageCheck}(g, c) = \text{Accepted} \iff \text{LineagePasses}(g) \wedge \text{gaps}(c) = [].$$

A rejection is located and its evidence is proved; an open result means exactly that the graph passes while gaps remain. (`lineage_check_reflect`, `defect_sound`, `lineage_check_rejected`, `lineage_check_open`)

Reflection is unconditional, without a well-formedness hypothesis, because well-formedness is itself checked and reflected: soundness and completeness of ancestors are needed to reflect acyclicity. Each of well-formedness, L1, L2 and L3 is separately reflected (`wf_defect_none_iff`, `check_L1_reflect`, `undisclosed_none_iff`, `check_L3_reflect`). An accepted check constructs a kernel-checked certificate whose graph and coverage are the ones checked (`issue_certificate_iff`, `issue_certificate_data`), and a rejection converts to a located construction failure for assessment.

11.5 The audit boundary

It is important to state exactly what is and is not proved. The Coq predicate and the Coq checker are related by Theorem 24. The *handwritten* OCaml audit that accompanies the case study is *not* proved equivalent to either; it implements the same stated conditions, and its agreement with the checked procedure is established by regression testing (Section 13), not by a theorem.

The extracted checker can take over the decision core: the handwritten code remains as input construction, diagnostic formatting, the case-study driver, and an independent comparison. This is preferable to verifying arbitrary handwritten OCaml against Coq after the fact. We have not yet replaced the handwritten decision core in the supplement, whose files are preserved unchanged.

12 Mechanisation and executable evidence

12.1 Development structure

The development comprises roughly three thousand lines of theory and eight hundred of examples. Its layers are kept separate:

1. the **legacy layer**: an earlier supplement, imported byte-for-byte, checksum-verified, and built as an independent library;
2. the **generic calculus**: the kernel structure, erasure, grounded-proper contexts, and obstructions;
3. the **compatibility layer**: proofs that the legacy seam, evolution and admissibility definitions instantiate the generic ones;

Table 2: Trust boundary and non-claims.

Checked mechanically	Coq 8.18.0; build with <code>coq_makefile</code> and <code>dune</code> ; <code>coqchk</code> on the whole project except the isolated classical file; <code>Print Assumptions</code> for every reported theorem; checksum verification of the legacy supplement.
Trusted	The Coq kernel and extraction mechanism; the encoding of $\mathbb{N}$ as OCaml integers in extracted code; the declared records themselves.
Not established	Completeness of declared lineage; accuracy of raw inputs; adequacy of the state model; legitimacy or competence of an authority; decidability of general obligations; equivalence of the handwritten OCaml audit with the checked one.

Table 3: Evidence fields and whether they survive extraction.

Component	Field	Survives
Exhibition certificate	record id, identifiers, process, retained record and evaluator, coverage, custodian	yes
	injectivity of identifiers; valuation factorisation proof	no
Lineage certificate	graph, coverage record with gaps, issued verdict	yes
	well-formedness, L1–L3 proofs	no
Construction certificate	seam inhabitant	yes
	grounding equations	no
Obstruction	constructor tag; located data (seam element, node id)	yes
	Prop-valued evidence	no
Assessment	verdict, open reasons, gaps	yes
Maintenance / authority	application-supplied data, via the summary interface	if in <code>Type</code>

4. the **assessment layer**: certificates, witnesses, located obstructions, assessments, the lineage checker, packed warrants;

5. **examples**: the separating models of Section 13.

The generic calculus is not replaced by the legacy material; each is preserved, and the compatibility layer proves that the original definitions instantiate the generic ones.

12.2 Trust boundary

12.3 Classical factorisation

The unrestricted functional converse of fibre factorisation needs classical choice and is isolated in one file that nothing else imports; the rest of the development is axiom-free. The constructive alternatives are the relational formulation and the finite theorem with a complete enumeration and decidable equality. The main development does not depend on the classical file: `coqchk` succeeds on the project without it and reports no axioms.

12.4 Extraction behaviour

Certificate data live in `Type`; their proofs live beside them. Extraction removes the proof fields and keeps the data.

We extracted the assessments of the examples to OCaml and wrote a consumer. For a certified assessment it reports the custodian (7), record id (1), lineage ground node (5) and audit verdict (`LineagePass`). For the copied model it reports a refutation at construction whose constructor names a lineage descent; for the maintenance model, a maintenance obstruction; for the model with both, both, in order. This is the concrete content of the claim that the formal

distinction survives compilation. It is materially stronger than attaching abstract propositions `Exhibited` and `LineageGrounded` to an ordinary relation, whose proofs would be erased.

12.5 Evidence summaries

Maintenance and authority evidence stay abstract in the generic library: the calculus should not impose one institutional record format. What it provides is a *summary interface*,

```
Class EvidenceSummary (A : Type) := { evidence_summary : A -> list EvidenceAtom }.
```

with atoms for records, custodians, ground nodes, intervals, authorities, maintenance events, expiry and revalidation. Applications supply instances, so an extracted consumer can inspect more than the exhibition and lineage components without the kernel fixing a schema. This is an observability extension, not a kernel correctness gap; the witness summary now includes maintenance and authority atoms.

13 Evaluation through separating examples

The examples are *separating models*, not empirical evaluation: each is a small finite structure that shows a distinction is real. They establish that the formal separations hold; they do not establish usability or scalability on production systems.

13.1 RQ1: Can ordinary agreement survive while grounding fails?

Copied coordinates. Extensional agreement succeeds and erasure yields ordinary substitutability; grounding fails at r_* ; assessment is `Refuted` with a construction obstruction (the declared lineage descends from a coordinate; independently, grounding fails at r_*). Ordinary agreement survives while grounded warrant fails (Theorems 6 and 11).

13.2 RQ2: Can structural naturality preserve agreement while losing grounding?

Maintenance failure. The squares commute; pairwise agreement persists at both stages and ordinary rewriting succeeds; grounding is lost at the transported element; assessment is `Refuted` with a maintenance obstruction naming the seam element (Theorem 16). The same problem with maintenance unassessed is `Open`, and with grounded valuations it is `Certified`: one problem, three outcomes.

13.3 RQ3: Can several failures be retained simultaneously?

Co-occurrence. A copied ground together with a lost grounding fails two obligations; both obstructions are reported, in a fixed order, with no first-error precedence discarding the second (Theorem 22). A further variant fails one obligation and leaves another open: the verdict is `Refuted` and the open obligation remains in the report (Theorem 18).

13.4 RQ4: Can different warrants erase to the same judgement?

Different warrants. Two witnesses, one within a single problem and one across problems with different declared lineage, connect the same endpoints and erase into the same judgement, while their summaries differ in record, custodian, coverage, authority, and lineage disposition (Theorem 7).

Table 4: Separating examples and outcomes.

Example	Ordinary relation	Assessment	Located evidence
Valid seam	holds	Certified	full witness: custodian, record, ground node, verdict
Copied coordinates	holds	Refuted	lineage descent; grounding failure at r_*
Missing lineage record	may hold	Open	<i>missing lineage record</i>
Partial lineage record	may hold	Open	<i>coverage gap</i> , gaps named
Dynamic failure	holds	Refuted	lost grounding at transported element
Co-occurrence	holds	Refuted	construction <i>and</i> maintenance obstruction
Failure + open	holds	Refuted	failure reported; open item retained in report

13.5 RQ5: Does missing evidence remain distinct from counterevidence?

A missing lineage record yields **Open**; a partial lineage record with declared gaps yields **Open** (coverage-qualified, under our declared standard); an explicit invalid lineage path yields **Refuted**. The checked audit distinguishes the last two on the same inputs: a passing graph with gaps is open, and a defect beats a gap.

13.6 Lineage audit: regression against the handwritten audit

The extracted, checked lineage procedure was compared with the handwritten OCaml audit of the case study, by capturing the latter’s printed output. On the five case-study records (live rows; backfill with a disclosed shared ancestor; backfill with the shared ancestor undisclosed; a copied seam valuation; a cyclic record) the two agree on all five, including the malformed cycle. On 3000 randomly generated records, including every malformation kind (duplicate names, undeclared parents, distinguished nodes not derived, relevant inputs not raw, and cycles), the two agree on all 3000. The generated records were 89 grounded, 1403 not grounded, and 1508 malformed.

Two remarks. First, this is a *test*: agreement on these inputs, not equivalence. Second, the harness discriminates. A deliberate mutation of the extracted side—dropping the d_B descent test from L1—was detected by the random test and not by the five case-study records; the case study alone would not have found that fault. The accepting path is exercised less than the rejecting paths, since few random records are grounded.

14 Relationship to existing formalisms

Equality and setoid rewriting. Ordinary rewriting identifies the relation that contexts must respect; generalised rewriting in type theory makes this systematic with respectful morphisms [10]. Grounded transport additionally retains the evidence that licenses the relation in the particular case.

Proper and respectful morphisms. **GroundedProper** refines **Proper**: it maps witnesses, not merely the truth of a relation. Erasure turns grounded-proper contexts into ordinary proper contexts (Section 6); the converse fails.

Proof-relevant relations. The calculus belongs to the family of proof-relevant relational structures, but adds application-specific certificate and assessment layers oriented towards lineage, maintenance and audit.

Table 5: Comparison with related formalisms.

Formalism	Retains	Does not retain (in general)
Setoid rewriting	the relation, context respect	how the relation was established
Rewriting logic / LTS	transition labels	checkable lineage, coverage, warrant
Proof-carrying code	semantic-preservation evidence	grounding independence, open obligations
Provenance	derivational history	when history licenses substitution
Assurance cases	arguments, defeaters	mechanised located obstructions
Grounded transport	seam, lineage, coverage, authority evidence; located failures; open items	real-world adequacy of declared records

Rewriting logic and labelled transitions. Rewriting logic [8] and labelled transition systems retain transition labels. A label alone need not contain independently checkable lineage, coverage and warrant evidence. Grounded witnesses can be read as richly certified transition labels; we do not develop this reading formally here.

Categorical transport. Identity, composition and structure-preserving erasure suggest a categorical interpretation [7]. We claim a category only up to the witness equivalence $\simeq$ of Section 4.4, for which the laws are proved; a stronger claim would require witness equality we do not have.

Proof-carrying systems and certified transformations. Grounded transport resembles proof-carrying code [9] and certified compilation [6] in carrying evidence across a transformation. Its distinctive concern is that the evidence includes grounding, lineage, open obligations and institutional parameters, not only semantic preservation.

Provenance systems. Provenance systems record derivational histories [3, 4]. Grounded transport adds claim-relative conditions determining whether that history licenses substitution: a history is not thereby a warrant.

Assurance cases. The certified/refuted/open distinction supports assurance cases that must retain defeaters and unresolved obligations rather than issue only success or failure [2].

Processual motivation. The settled endpoints correspond to coordinate description; the witness-bearing transition retains something of the production of that agreement [12]. The formal calculus does not encode any ontology and should not be read as doing so. It formalises one methodological consequence: endpoint agreement does not exhaust the conditions under which a processual transition warrants substitution. The full treatment, including the taxonomy of warrant debt, is in a companion manuscript [1].

15 Limitations

Declared-graph limitation. A lineage certificate proves properties of the *declared* graph. It does not prove that the graph contains every real production path, and the audit cannot detect a production path the record omits.

Raw-input limitation. The calculus can identify claim-relevant raw inputs without proving that they accurately record the concrete process.

Institutional limitation. Authority and custodianship can be represented as data, but their legitimacy and competence are not established internally; the rule system is an input. Exhibition conditions concerning the declared process, coverage and custodian are recorded, not verified.

Decidability limitation. General transport obligations need not be decidable. The open assessment is essential, not an implementation inconvenience.

Coverage limitation. Preservation on the image of an evolution does not certify new target states outside that image (Theorem 15).

The endpoint theorem is extensional. The equality concluded by seam transport needs only extensional agreement. Grounding strengthens its warrant, not its algebraic conclusion (Section 7).

Composition in the concrete instance. In a single seam a non-identity witness is one crossing, so composition of non-trivial witnesses is degenerate there; its content appears through contexts and evolution.

Witness algebra modulo an equivalence. Identity and associativity hold up to the crossing-sequence equivalence, not by literal equality (Section 4.4).

The audit boundary. The Coq checker is proved to reflect its specification; the handwritten OCaml audit is tested against it, not proved equivalent (Sections 11 and 13). The decision core of the supplement has not yet been replaced by the extracted checker. Assessment components for maintenance and authority are abstract in the generic library.

Boolean valuations. Valuations are Boolean; a general value space with a value transport is used only in the statement of valuation naturality.

No automated tactic yet. The contribution is semantic and mechanised, not a packaged rewriting tactic.

No empirical claim. The examples demonstrate logical separations, not usability or scalability on production systems.

16 Conclusion

The central distinction is between

$$\text{same endpoint judgement} \neq \text{same warrant}.$$

Grounded transport carries evidence; composition and grounded-proper contexts preserve it; erasure yields ordinary substitutability and loses information irreversibly; located obstructions explain failed transport; open assessments preserve unresolved obligations, and a refutation does not discard them; and extraction retains the evidential data that downstream audits require.

Ordinary rewriting answers whether one expression may replace another under a relation. Grounded transport additionally answers through which process, under which lineage, within what coverage, and with which remaining obligations that replacement is warranted. Erasure

recovers ordinary substitutability, but ordinary substitutability cannot reconstruct the warrant that was erased.

The calculus does not make declarations true. It makes precisely explicit what has been declared, checks what can be checked, and reports what has not been resolved.

References

- [1] Anonymous. The work of exactness: Whitehead, warrant debt, and computational systems. Companion manuscript, under review; supplement archived with this development, 2026.
- [2] Robin Bloomfield and John Rushby. Assurance 2.0: A manifesto. arXiv:2004.10474, 2020.
- [3] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory — ICDT 2001*, volume 1973 of *LNCS*, pages 316–330, 2001.
- [4] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.
- [5] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.
- [6] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- [7] Saunders Mac Lane. *Categories for the Working Mathematician*. Springer, 2 edition, 1998.
- [8] José Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science*, 96(1):73–155, 1992.
- [9] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 106–119, 1997.
- [10] Matthieu Sozeau. A new look at generalized rewriting in type theory. *Journal of Formalized Reasoning*, 2(1):41–62, 2009.
- [11] The Coq Development Team. The coq proof assistant, version 8.18.0. <https://coq.inria.fr>, 2023.
- [12] Alfred North Whitehead. *Process and Reality*. Macmillan, 1929.

A Theorem ledger

The main text is organised around the following theorem spine. Each row names the Coq identifier(s) in the accompanying development; helper lemmas are in the supplement. All listed results are closed under the global context (no axioms), except where noted.

Table 6: Theorem spine and its mechanisation.

#	Result	Coq identifier(s)	Text
T1	Grounded reflexivity	<code>grounded_transport_refl</code>	Theorem 1
T2	Grounded composition	<code>grounded_transport_compose</code>	Theorem 2
T2'	Witness algebra modulo $\simeq$	<code>WEquiv_id_left</code> , <code>WEquiv_id_right</code> , <code>WEquiv_assoc</code> , <code>WEquiv_compose</code>	Theorem 3
T3	Erasure soundness	<code>erase_sound</code> , <code>rpath_sound</code> , <code>erasure_sound</code>	Theorem 4
T4	Erased preorder	<code>erased_PreOrder</code>	Theorem 5
T5	Erasure non-reflection	<code>erasure_not_reflecting</code> , <code>copied_ungrounded_agreement</code>	Theorem 6
T6	Same endpoints, different warrants	<code>same_endpoints_different_warrants</code> , <code>packed_warrants_same_endpoints</code>	Theorem 7
T7	Grounded-proper composition	<code>gp_id</code> , <code>gp_comp</code>	Theorem 8
T8	Nested contextual preservation	<code>contextual_preservation_nested</code>	Theorem 9
T9	Grounding erases to extensional agreement	<code>original_grounding_erases_to_extensional</code>	Theorem 10
T10	Pairwise agreement $\not\Rightarrow$ grounding	<code>pairwise_agreement_not_grounding</code>	Theorem 11
T11	Original seam transport recovered	<code>original_seam_transport_from_generic</code>	Corollary 13
T12	Naturality preserves grounding on the image	<code>natural_valuations_preserve_grounding_on_image</code>	Theorem 14
T13	Global preservation under coverage	<code>surjective_evolution_preserves_all_grounding</code>	Theorem 15
T14	Extensional persistence, grounding loss	<code>dynamic_failure_visible</code>	Theorem 16
T15	Lineage-sensitive obstruction	<code>not_proper_of_distinguished</code> , <code>observational_not_grounded</code>	Theorem 17
T16	Certification soundness	<code>certified_gives_transportable</code> , <code>assemble_status</code>	Theorem 19
T17	Refutation soundness	<code>obstructions_refute</code> , <code>each_obstruction_refutes</code>	Theorem 20
T18	Open-status characterisation	<code>assemble_status</code> , <code>underdetermined_iff</code> , <code>assemble_open_nonempty</code>	Theorem 21
T19	Complete failure reporting	<code>assemble_refuted_reports_all</code> , <code>report_accounts_for_all</code> , <code>report_verdict_agrees</code>	Theorems 18 and 22
T20	Lineage-check reflection	<code>lineage_check_reflect</code> , <code>ancestors_iff</code>	Theorems 23 and 24
—	Functional converse of fibre factorisation (classical)	<code>constant_factors</code> (isolated file)	Section 9

Supplement contents. Complete Coq listings; the `Print Assumptions` transcripts; the legacy checksum tables; build configuration; the extracted OCaml output and its consumers; and the regression harness for the lineage audit. The tactic implementation is not included, as none exists yet.